

GUIA DE APRESENTAÇÃO

# Maresia Log

Documento de apoio para explicar o projeto, a proposta de negócio e a parte técnica em AngularJS.

Projeto acadêmico de Programação Avançada

Aplicação principal: **AngularJS 1.8.2**

Desafio extra: **Vue.js 3 + Vite**

# 1. Visão Geral do Projeto

---

A **Maresia Log** é uma empresa fictícia de logística refrigerada. A ideia do sistema é apoiar pequenos produtores de alimentos frescos em Salvador e Região Metropolitana, conectando esses produtores a restaurantes, cafés e mercados de bairro.

O projeto apresenta uma interface web com rotas, coletas, entregas, controle de temperatura, alertas operacionais, filtros por bairro e componentes de interação.

## Frase para explicar:

"O projeto é um protótipo front-end de uma plataforma de logística refrigerada. Ele simula rotas, coletas, alertas e dados operacionais para demonstrar AngularJS, componentes reutilizáveis e interação com o usuário."

## Para que serve?

- Visualizar rotas refrigeradas da semana.
- Filtrar rotas por bairro.
- Simular uma coleta com responsável, horário e temperatura.
- Agendar uma rota piloto com produtor, carga e janela final.
- Visualizar alertas operacionais gerados pelas condições do sistema.
- Mostrar feedback ao usuário por notificações toast.

## Isso é um MVP?

Pode ser explicado como um **MVP front-end acadêmico** ou um **protótipo funcional de interface**. Ele não é um sistema de produção completo, porque não tem backend, login, banco de dados real ou API. Mesmo assim, ele valida a ideia principal: uma interface simples para acompanhar rotas refrigeradas.

## 2. O Que Foi Implementado

| Funcionalidade         | Como funciona no projeto                                                                                                                                                                          |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Filtro por bairro      | O menu suspenso altera o bairro selecionado e a lista mostra apenas as rotas daquele bairro.                                                                                                      |
| Simular coleta         | Abre um formulário. O usuário escolhe rota, responsável, temperatura e janela de coleta. Ao confirmar, a rota muda de status, o resumo operacional é atualizado e a temperatura entra no gráfico. |
| Agendar rota piloto    | Abre um formulário para criar uma nova rota com bairro, produtor, carga, temperatura prevista e janela final.                                                                                     |
| Ver alerta operacional | Mostra alertas calculados pelo sistema, como temperatura acima do limite, coletas pendentes e rotas com ocorrência térmica.                                                                       |
| Gráfico de temperatura | As barras são renderizadas a partir das leituras salvas no estado local. Novas coletas e rotas acrescentam dados ao gráfico.                                                                      |
| Persistência           | Os dados são salvos no <code>localStorage</code> do navegador, então continuam existindo depois de atualizar a página.                                                                            |

## 3. Tecnologias Usadas

| Tecnologia      | Uso no projeto                                                                                         |
|-----------------|--------------------------------------------------------------------------------------------------------|
| HTML5           | Estrutura da página, formulários, seções, botões e conteúdo.                                           |
| CSS3            | Layout, responsividade, cores, cards, modal, animações e aparência geral.                              |
| AngularJS 1.8.2 | Framework principal da aplicação, usado com controller, services, directives e bindings.               |
| localStorage    | Persistência local no navegador, usada para guardar rotas, temperaturas, alertas e resumo operacional. |
| Vue.js 3 + Vite | Versão extra do mesmo projeto, feita como desafio adicional.                                           |

**Importante:** o projeto não usa backend. Por isso, os dados não vão para um servidor. Eles ficam salvos somente no navegador do usuário, via `localStorage`.

## 4. Estrutura de Pastas

| Arquivo/Pasta                                  | Função                                                                                              |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>angularjs-maresia/index.html</code>      | Ponto de entrada da aplicação AngularJS. Contém o HTML da página e chama os scripts.                |
| <code>assets/css/styles.css</code>             | Todo o visual da aplicação: layout, cores, cards, modais, botões e responsividade.                  |
| <code>src/app.js</code>                        | Cria o módulo raiz do AngularJS: <code>maresiaApp</code> .                                          |
| <code>src/controllers/mainController.js</code> | Controla a lógica principal: filtros, formulários, rotas, gráfico, alertas e chamadas aos serviços. |
| <code>src/services/dataService.js</code>       | Fornece dados iniciais, carrega e salva dados no <code>localStorage</code> .                        |
| <code>src/services/toastService.js</code>      | Serviço de notificações temporárias.                                                                |
| <code>src/directives/</code>                   | Componentes reutilizáveis AngularJS: acordeão, dropdown, paginação, progresso, abas e toast.        |
| <code>vue-maresia-desafio/</code>              | Versão extra em Vue.js, com componentes equivalentes.                                               |

## 5. Como o AngularJS Entra no Projeto

O projeto usa **AngularJS**, que é a versão 1.x do Angular. Ele é diferente do Angular moderno. O AngularJS trabalha muito com **controllers**, **\$scope**, **directives** e **data binding**.

### Fluxo básico

1. O arquivo `index.html` declara `ng-app="maresiaApp"`.
2. O arquivo `src/app.js` cria o módulo AngularJS chamado `maresiaApp`.
3. O `body` usa `ng-controller="MainController"`.
4. O `MainController` coloca dados e funções no `$scope`.
5. O HTML lê esses dados com `{{ }}` e reage a eventos com diretivas como `ng-click`, `ng-repeat` e `ng-submit`.

#### Explicação simples:

"O AngularJS liga o JavaScript ao HTML. Quando um dado muda no controller, a tela atualiza. Quando o usuário clica em um botão ou envia um formulário, o Angular chama uma função no controller."

## 6. Conceitos que o Professor Pode Perguntar

---

### Módulo

O módulo é o contêiner principal da aplicação AngularJS. Neste projeto ele é criado em

```
src/app.js: angular.module('maresiaApp', []).
```

### Controller

O controller é responsável por controlar o estado da tela e responder às ações do usuário. No projeto, o `MainController` guarda rotas, bairro selecionado, dados do gráfico, formulários e funções como `confirmPickup()` e `confirmPilotRoute()`.

### \$scope

O `$scope` é o objeto que faz a ponte entre o controller e o HTML. Quando o controller define `$scope.rotas`, o HTML consegue usar essa lista com `ng-repeat`.

### Service / Factory

Services guardam lógica reutilizável. O `dataService` centraliza dados e persistência no `localStorage`. O `toastService` centraliza mensagens de feedback.

### Directive

Diretiva é a forma do AngularJS criar componentes reutilizáveis ou comportamentos no HTML. Neste projeto, componentes como `<m-dropdown>`, `<m-pagination>` e `<m-toast>` são diretivas.

### Data Binding

Data binding é a ligação entre dado e interface. Exemplo: `{{ operationSummary.currentTime }}` mostra na tela o valor atual desse dado.

### Two-way Binding

É quando a interface e o JavaScript compartilham o mesmo valor. Exemplo: em formulários, `ng-model="pickupForm.responsavel"` atualiza o objeto do controller conforme o usuário digita.

### Dependency Injection

AngularJS injeta dependências automaticamente no controller. O projeto usa

```
MainController.$inject = ['$scope', 'dataService', 'toastService']
```

 para declarar o que o controller precisa receber.

### Digest Cycle

É o mecanismo interno do AngularJS que observa mudanças no `$scope` e atualiza a tela. Você não precisa chamar `render` manualmente; o AngularJS faz essa sincronização.

## 7. Componentes Obrigatórios

| Componente         | Arquivo                     | Explicação curta                                                 |
|--------------------|-----------------------------|------------------------------------------------------------------|
| Acordeão           | <code>mAccordion.js</code>  | Mostra perguntas frequentes; abre e fecha respostas.             |
| Paginação          | <code>mPagination.js</code> | Divide a lista de rotas em páginas.                              |
| Barra de progresso | <code>mProgress.js</code>   | Mostra o percentual de implantação da rota piloto.               |
| Abas               | <code>mTabs.js</code>       | Organiza texto por público: produtores, restaurantes e operação. |
| Dropdown           | <code>mDropdown.js</code>   | Menu suspenso usado para filtrar rotas por bairro.               |
| Toast              | <code>mToast.js</code>      | Mostra mensagens temporárias de sucesso, erro ou informação.     |

## 8. Regras de Negócio Simuladas

### Coleta

Uma coleta registra responsável, rota, temperatura e janela. Se a temperatura for alta, a rota pode virar alerta térmico.

### Rota piloto

Cria uma rota nova para testar atendimento com produtor, carga e janela final.

### Alerta operacional

É calculado a partir do estado: temperatura alta, média elevada, rotas com alerta térmico ou coletas pendentes.

### Gráfico

Mostra leituras de temperatura salvas localmente. Cada nova ação pode adicionar uma leitura.

## 9. Perguntas Prováveis e Respostas Curtas

---

### O projeto tem banco de dados?

Não. Ele usa `localStorage`, que é armazenamento local do navegador.

### O projeto é Angular moderno?

Não. É **AngularJS 1.8.2**, a versão antiga, baseada em controllers, `$scope`, services e directives.

### Por que usar services?

Para separar responsabilidades. O controller não precisa guardar sozinho a lógica de dados e notificações.

### Por que usar directives?

Para transformar partes repetidas da interface em componentes reutilizáveis.

### Qual é a diferença entre controller e directive?

O controller orquestra a página inteira. A directive cuida de um componente específico da interface.

### O alerta operacional é criado pelo usuário?

Não. Ele é visualizado pelo usuário, mas é gerado pelo sistema com base nas condições da operação.

### O que acontece ao atualizar a página?

Os dados alterados continuam salvos porque o projeto usa `localStorage`.

## 10. Roteiro de Fala

**Começo:** "Esse projeto é a Maresia Log, um protótipo front-end de uma empresa fictícia de logística refrigerada para pequenos produtores."

**Funcionalidade:** "Na tela eu consigo ver rotas, filtrar por bairro, simular coleta, agendar rota piloto e visualizar alertas operacionais. Os dados ficam salvos no `localStorage`."

**Tecnologia:** "A aplicação principal foi feita em AngularJS 1.8.2. O HTML se conecta ao MainController pelo ng-controller, o controller controla o estado pelo `$scope`, os services centralizam dados e notificações, e as directives implementam os componentes obrigatórios."

**Fechamento:** "Então é um MVP/protótipo funcional de interface: não tem backend, mas demonstra a experiência e a lógica principal do sistema."